Les Tableaux Unidimensionnels et le Passage par Adresse en Langage C

Thème principal : Programmation Impérative - Structures de données et Pointeurs

Synthèse exécutive : Ce cours détaille le fonctionnement interne des tableaux unidimensionnels en langage C. Il expose la disposition contiguë des données en mémoire vive, l'accès direct en temps constant O(1) via la formule de calcul d'adresse, ainsi que le mécanisme fondamental d'Array Decay (dégradation du tableau en pointeur lors de la transmission aux fonctions). L'objectif est d'apprendre à manipuler efficacement des séquences de données homogènes et à transmettre des conteneurs à des fonctions sans dégrader les performances ni dupliquer inutilement la mémoire.

Définition, allocation contiguë et calcul d'adresse mémoire

En langage C, la déclaration d'un tableau statique comme `int tab[5];` ordonne au compilateur de réserver un bloc mémoire d'un seul tenant en mémoire vive (la pile). La taille totale réservée est exactement égale à `N * sizeof(type)`, soit pour 5 entiers de 4 octets : `5 * 4 = 20 octets consécutifs`. L'accès aux éléments via la syntaxe crochet `tab[i]` s'appuie sur une formule mathématique d'arithmétique système : `Adresse(tab[i]) = Adresse_Base + (i * sizeof(type))`. L'indice `i` représente le décalage (offset) par rapport au premier élément. Ainsi, pour l'élément initial, le décalage est 0, d'où `tab[0]`. Cette propriété garantit une complexité temporelle algorithmique en temps constant O(1), rendant les tableaux extrêmement rapides à la lecture et à l'écriture.

- Tableau unidimensionnel : Séquence homogène d'éléments de même type stockés sous une taille fixe prédéterminée.
- Contiguïté mémoire : Emplacement mémoire où toutes les cases se suivent strictement sans aucun espace ou trou entre elles.
- Indice (Index) : Position relative d'un élément, débutant impérativement à 0 jusqu'à N-1 pour N éléments.
- Accès en O(1) : Temps d'accès direct égal pour tout élément du tableau, indépendamment de sa position.

Cas concret enseignant : *Le professeur a utilisé l'analogie d'un meuble de rangement vertical comportant des tiroirs rigoureusement identiques et collés les uns aux autres, numérotés de 0 à N-1. Si l'on connaît le sol (l'adresse de base du meuble) et la hauteur exacte d'un tiroir, on calcule instantanément la hauteur du tiroir `i` sans avoir besoin de mesurer ou d'ouvrir tous les tiroirs précédents un par un.*

Alignement, pointeurs et mécanisme d'Array Decay

Lorsqu'un tableau est transmis à une fonction (ex: `ma_fonction(tab)`), le langage C interdit le passage par valeur (copie intégrale de la structure). Le nom du tableau `tab` subit une 'dégradation' (Array Decay) et se transforme immédiatement en un pointeur `type*` pointant vers la première case mémoire `&tab;[0]`. Ce choix garantit des performances optimales en évitant de dupliquer des mégaoctets de données. Cependant, cela entraîne une conséquence majeure : à l'intérieur de la fonction recevant le pointeur, le type original 'tableau de N éléments' est perdu. La fonction n'a aucun moyen natif de savoir combien de cases comporte le tableau. Il est donc impératif de transmettre la taille effective du tableau en paramètre complémentaire (souvent un `int taille` ou `size_t taille`). Enfin, modifier le tableau dans la fonction altère directement le tableau de la fonction appelante.

- Array Decay : Conversion implicite et automatique du nom d'un tableau en un pointeur vers son premier élément `&tab;[0]` dès qu'il est passé à une fonction.
- Passage par adresse : Transmettre l'emplacement mémoire d'une donnée plutôt que d'en faire une copie intégrale.
- Arithmétique des pointeurs : Équivalence stricte entre l'écriture pointeur `*(tab + i)` et l'écriture crochets `tab[i]`.
- Perte d'information de taille : Une fonction recevant un tableau ne peut pas déterminer dynamiquement combien d'éléments il contient.

Cas concret enseignant : *Pour faire assimiler cette notion, le professeur a expliqué que transmettre un tableau à une fonction ne revient pas à construire une réplique exacte de sa maison dans le jardin de son ami (passage par valeur), mais plutôt à lui donner les coordonnées GPS précises de sa propre maison (passage par adresse). L'ami utilise ces coordonnées pour venir repeindre les murs directement sur l'édifice original.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou affiche des valeurs absurdes lorsque j'essaie de lire ou d'écrire dans tab[5] alors que mon tableau a été déclaré avec une taille de 5 (int tab[5];) ?
Réponse du professeur : En langage C, les indices d'un tableau de taille N vont de 0 à N-1. Pour un tableau de 5 éléments, les indices valides sont donc 0, 1, 2, 3 et 4. Essayer d'accéder à tab[5] revient à lire ou écrire en dehors de la zone mémoire allouée (c'est un dépassement de tampon ou buffer overflow). Le programme accède à une case mémoire adjacente non contrôlée, ce qui provoque un comportement indéfini (undefined behavior), un plantage (Segmentation Fault) ou une corruption de données.

Question étudiant : Est-ce qu'on peut calculer la taille d'un tableau à l'intérieur d'une fonction avec l'expression sizeof(tab) / sizeof(tab[0]) ?
Réponse du professeur : Non, c'est une erreur classique fatale. À l'intérieur de la fonction, à cause du phénomène d'Array Decay, `tab` n'est plus un tableau mais un simple pointeur (`int*`). L'expression `sizeof(tab)` renverra donc la taille du pointeur en mémoire (soit 8 octets sur une architecture 64 bits) et non la taille totale du tableau. C'est pour cette raison impérative qu'il faut toujours passer la taille du tableau en paramètre distinct lors de son appel.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en C une fonction nommée `remplir_et_calculer_moyenne` qui prend en paramètres un tableau d'entiers et sa taille effective. La fonction doit demander à l'utilisateur d'entrer chaque note au clavier pour remplir le tableau, puis calculer et retourner la moyenne arithmétique exacte des notes saisies sous la forme d'un nombre à virgule (`double`).

Résolution pas à pas :
1. Définition du prototype : `double remplir_et_calculer_moyenne(int tab[], int taille);`. 2. Vérification de sécurité : S'assurer que `taille > 0` pour éviter une division par zéro. 3. Initialisation de la somme : Déclarer une variable `double somme = 0.0;` pour stocker l'accumulateur. 4. Parcours par boucle `for` : Utiliser une variable `int i` allant de 0 à `taille - 1`. 5. Saisie utilisateur & accumulation : Pour chaque indice, utiliser `scanf("%d", &tab;[i]);` et ajouter immédiatement la valeur saisie à `somme`. 6. Calcul final et retour : Diviser `somme` par `taille` et renvoyer la valeur `double` obtenue.

Résultat : #include double remplir_et_calculer_moyenne(int tab[], int taille) { if (taille <= 0) { return 0.0; } double somme = 0.0; for (int i = 0; i < taille; i++) { printf("Saisir l'élément [%d] : ", i); scanf("%d", &tab;[i]); somme += tab[i]; } return somme / taille; } // Exemple de test dans main() avec un tableau de taille 3 : // Valeurs saisies : 12, 14, 16 -> Moyenne retournée = 14.000000

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'indice du dernier élément accessible d'un tableau d'entiers déclaré par int t[10]; ?

R : L'indice 9, car la numérotation commence toujours à 0 (les indices valides vont de 0 à 9).

Q : Que contient exactement le nom d'un tableau lorsqu'il est employé seul sans crochets dans un programme C ?

R : Il contient l'adresse mémoire du premier élément du tableau, ce qui équivaut à &tab;[0].

Q : Pourquoi la signature d'une fonction manipulant un tableau C doit-elle obligatoirement inclure la taille du tableau en paramètre ?

R : À cause de l'Array Decay : lorsqu'un tableau est passé à une fonction, il se dégrade en pointeur et perd son information de taille. La fonction ne connaît donc pas le nombre d'éléments sans ce paramètre explicite.